FREE LEARNING MATERIAL · R FOR DATA ANALYSIS

R for Data Analysis

A step-by-step R course for monitoring, evaluation and research teams: from your first calculation to cleaning and editing survey data. Each lesson has a short video, the R code, and the full narration.

23 lessons | About 3 minutes each | Beginner friendly

Prepared by

Aqil Shah Ayaz

Data Analyst · Monitoring, Evaluation and Learning Expert

Free learning material

aqilshah-researcher.github.io

INSIDE THIS WORKBOOK

How to use it

Watch each video on the website, then type the code yourself in RStudio. Typing is how the skills stick. Read the narration if you prefer to learn by reading.

Practice data
Use any small Excel survey file of your own, with columns such as province, sex of household head, marital status and household size. Never practice on data with real names or ID numbers.

01 R basics: Your first calculations in R, Text, numbers and TRUE or FALSE, Variables and your first vector, Comparison operators, Calling functions: sort, sum, length, mean, median **3**

02 Getting ready: Clean your console and environment, Install and load packages, Set your working directory, Open an Excel file and inspect it **5**

03 Working with data: The pipe and select(): build a smaller table, Remove columns and rows, Rename columns, two ways, Sort rows with arrange(), Filter rows with filter(), Filter with several conditions, Combine tables with bind_cols() and bind_rows() **7**

04 Cleaning and recoding: Recode labels to numbers with if_else(), Group numbers into ranges with cut() **9**

05 Editing data: A new dataset: load, view and select, Select columns by position with [,], Check counts with table() and remove rows safely, Change one cell, or many cells at once, attach(): type column names with ease **11**

R FOR DATA ANALYSIS · R BASICS · LESSON 1

01 Your first calculations in R

Meet RStudio and use R as a calculator, one line at a time.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Find the four panes of RStudio
- Run a line of code with Control and Enter
- Read the answer in the console

The R code in this lesson

```
# Basic calculation
2+5
574-4
2+5/3+10*8-9
```

What you will hear (times count from the end of the intro)

Intro Welcome to your first R lesson. Take a deep breath, there is no rush. In this lesson, you will meet RStudio, write your very first lines of code, and see R give you the answers. We will go step by step, together.

0:00 This is RStudio. On the top left is the script, where we write our code. Below it is the console, where R shows the results. On the right, you see the environment, and your files.

0:15 Let's start with something simple. I write a small note first, with a hash sign: Basic Calculation. Anything after a hash is a comment. R does not run it. It is only there to help us remember.

0:32 Now our first line: two plus five. Put your cursor on the line, and press Control and Enter. On a Mac, it is Command and Enter.

0:43 Look at the console. R says seven. Wonderful, you just ran your first piece of code!

0:50 Next line: five hundred seventy four minus four. Run it the same way. The answer is five hundred seventy.

0:58 Now a longer one: two plus five divided by three, plus ten times eight, minus nine. R follows the usual math order. It divides and multiplies first, then adds and subtracts. The answer is seventy four point six six.

1:25 Notice how every result in the console starts with a one in brackets. That just means this is the first value of the answer. Nothing to worry about.

1:35 Next, I add a new heading: R Data Types. Under it, three more notes: character, numeric, and logical. These are the three kinds of values we will meet first.

2:10 Let's begin with the character type. A character is simply text, like a name. Text in R always goes inside quotation marks.

2:23 Here I type a name inside quotes, as our first example. In the next lesson, we will look at each data type closely. You did great today!

R FOR DATA ANALYSIS · R BASICS · LESSON 2

02 Text, numbers and TRUE or FALSE

The three basic data types in R, and the difference between one and two equals signs.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Recognize character, numeric and logical values
- Put text inside quotation marks
- Compare two values with a double equals sign

The R code in this lesson

```
# Character: text goes in quotes
"Ahmad"

# Numeric
30

# Logical: TRUE or FALSE
50 == 50   # TRUE
50 == 25   # FALSE
# 25 = 50 gives an error: one = means assign, not compare
```

What you will hear (times count from the end of the intro)

Intro Every value in R has a type. In this lesson, you will meet the three most common ones: text, numbers, and TRUE or FALSE. You will also see a small mistake that almost everyone makes at the start, and how to fix it. Let's take it slowly.

0:00 Here are our three notes from before: character, numeric, and logical. Let's look at each one.

0:10 First, character. Under the note, I wrote a name inside quotation marks. The quotes tell R: this is text. Without quotes, R would think it is the name of something else.

0:55 Next, numeric. A number, like thirty for an age, needs no quotes. R knows it is a number, and you can do math with it.

1:35 Now the logical type. Logical values are only TRUE or FALSE. We get them when we ask R a question, like: is twenty five equal to fifty?

1:52 I typed twenty five, one equals sign, fifty. And R shows an error in red. Please don't worry when you see red. It is R's way of helping us.

2:08 The problem is the single equals sign. In R, one equals sign means assign a value. To compare two values, we need two equals signs.

2:25 So I write fifty, two equals signs, fifty. Run it, and R says TRUE. Fifty is equal to fifty.

2:40 Then fifty, two equals signs, twenty five. R says FALSE. They are not equal.

2:55 So remember: text in quotes, numbers without quotes, and two equals signs to compare. Very well done.

R FOR DATA ANALYSIS · R BASICS · LESSON 3

03 Variables and your first vector

Save values with the assignment arrow, then group them together in a vector.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Save a value with the arrow <-
- Find your variables in the Environment pane
- Combine values into a vector with c()

The R code in this lesson

```
# Assign values with <-
name <- "Ahmad"
age <- 30
location <- "USA"

# Print them
name
age
location

# A vector lists related values with c()
demographic <- c(name, age, location)
demographic
```

What you will hear (times count from the end of the intro)

Intro Now we will teach R to remember things. In this lesson, you will learn to save values in variables with the assignment arrow, find them in the Environment pane, and group related values together in a vector. Let's go gently, one line at a time.

0:00 I add a note: assigning a variable. For this, we use the assignment operator. It is a small arrow: a less than sign, and a dash.

0:12 First line: name, arrow, and a name in quotes. Run it. Look at the Environment pane on the right. A new variable, name, has appeared. R remembers it now.

0:22 Next, age, arrow, thirty. And location, arrow, USA.

0:30 Oh, look at the console: object USA not found. I forgot the quotation marks. Without quotes, R looks for a variable called USA. So I add the quotes, run it again, and now it works.

0:50 To see a value, just type the variable name and run it. Name gives the name. Age gives thirty. Location gives USA.

1:20 Now, a new idea: the vector. A vector keeps related values together in one place. We make it with the letter c and brackets. The c stands for combine.

1:58 I write: demographic, arrow, c, and inside the brackets, name, age, and location, separated by commas.

2:30 Run it, and in the Environment pane you see demographic, with three values. R even tells us it is character, because it holds text.

2:50 One thing to notice: thirty now appears in quotes. A vector keeps one type, so R turned the number into text to match. Good to remember! You are doing really well.

R FOR DATA ANALYSIS · R BASICS · LESSON 4

04 Comparison operators

Ask R yes or no questions with less than, greater than, equal and not equal.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Use the six comparison operators
- Read TRUE and FALSE answers
- Know the difference between == and !=

The R code in this lesson

```
# Comparison operators
# Less than: <    Greater than: >
# Less than or equal to: <=   Greater than or equal to: >=
# Is equal to: ==   Is not equal to: !=
25 < 50    # TRUE
25 > 50    # FALSE
25 <= 50   # TRUE
25 >= 50   # FALSE
25 == 50   # FALSE
25 != 50   # TRUE
```

What you will hear (times count from the end of the intro)

Intro Data analysis is full of small questions: is this bigger, is this smaller, are these the same? In this lesson, you will learn the six comparison operators in R, and how R answers each question with TRUE or FALSE. It is easier than it sounds. Let's begin.

0:00 Here I wrote a short list of notes, so we can see all six operators together. Less than, greater than, less than or equal to, greater than or equal to, is equal to, and is not equal to.

0:20 Let's try them one by one, with the same two numbers: twenty five and fifty.

0:26 Twenty five, less than, fifty. Run it. R says TRUE. Twenty five is smaller.

0:32 Twenty five, greater than, fifty. FALSE. Of course, it is not bigger.

0:40 Now less than or equal to. We write the less than sign, then the equals sign. Twenty five, less than or equal to, fifty: TRUE.

0:48 Greater than or equal to: FALSE.

0:55 Is equal to, with two equals signs: FALSE, because the numbers are different.

1:02 And is not equal to, with an exclamation mark and an equals sign. Twenty five is not equal to fifty: TRUE.

1:15 Take a moment to read the answers in the console. Every question gives TRUE or FALSE. We will use these questions a lot later, when we filter data.

1:30 Next, I add a new note: calling a function. A function is a ready-made tool in R. You give it some values, and it does the work for you.

1:50 Our first function is sort. Inside the brackets, I put a vector of numbers with c. Run it, and R lists the numbers from smallest to largest.

2:20 Then sum, which adds them all up. The answer is five thousand five hundred ninety four. We will meet more functions in the next lesson. Well done!

05 Calling functions: sort, sum, length, mean, median

Use R's built-in functions to sort and summarize a list of numbers.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Call a function with values in brackets
- Summarize numbers with sum, length, mean and median
- Understand why mode() gives a surprise

The R code in this lesson

```
x <- c(1,5,3,5,6,4,3,65,3,3,4,6,8,5433,45)
sort(x)     # smallest to largest
sum(x)      # 5594
length(x)   # 15 values
mean(x)     # 372.93
median(x)   # 5
mode(x)     # "numeric": mode() tells the type, not the most common value
```

What you will hear (times count from the end of the intro)

Intro Functions are the heart of R. In this lesson, you will use five friendly functions to sort and summarize numbers, and you will see one function that does something surprising. Let's explore them together, slowly.

0:00 Here are the functions from our list. First, sort. It puts our numbers in order, from the smallest to the largest. In the console, we see one, three, three, all the way up to five thousand four hundred thirty three.

0:15 Next, sum. Notice, sum does not even need c. You can give it the numbers directly. The total is five thousand five hundred ninety four.

0:30 Now length. Length counts how many values we have. I type length, open the brackets, and inside, c with the same numbers.

1:00 Run it. The answer is fifteen. We have fifteen numbers.

1:20 Next, mean, which is the average. And median, the middle value when the numbers are sorted.

1:35 When I first ran mode, I got an error: unused arguments. That is because I forgot c around the numbers. So I add c, and run it again.

2:05 The mean is three hundred seventy two point nine three. That is high, because one number, five thousand four hundred thirty three, pulls the average up.

2:20 The median is only five. When a few values are very big, the median gives a better picture of the typical value. That is a useful lesson for any data work.

2:30 And mode? It says numeric. In R, mode tells you the type of the data, not the most common value. A small surprise, and a good one to remember.

2:48 Finally, I scroll back to the top of our script. Look how much we wrote today. Calculations, data types, variables, vectors, comparisons, and functions. Excellent work!

R FOR DATA ANALYSIS · GETTING READY · LESSON 1

06 Clean your console and environment

Start fresh: clear the console and remove old values from the environment.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Clear the console with Control and L
- Remove one value with remove()
- Remove several values at once

The R code in this lesson

```
# Clear the console: press Ctrl + L (this is a keyboard shortcut, not code)

# Remove one value from the Global Environment
remove(age)

# Remove several values at once
remove(City, demographic, Detail, location, name)
```

What you will hear (times count from the end of the intro)

Intro Welcome back! Before we start new work, it helps to clean up. In this lesson, you will learn to clear the console, and to remove old values from the environment, one at a time or many at once. A clean workspace is a calm workspace.

0:00 This is our new session. On the right, the environment still holds the values from last time. In the console, we see our old results too.

0:13 First, the console. I write a note: to remove history from the console. The shortcut is Control and L.

0:33 Careful here: I typed Control L as code and ran it. R says: object not found. That is because Control L is not code. It is a keyboard shortcut. Just click in the console, and press Control and L.

0:45 See? The console is clean now. The script and the environment are untouched.

0:55 Next, the environment. I write a note: to remove a data or value from the Global Environment. The function is called remove.

1:20 Inside the brackets, I write the name of the value. Remove age. Run it, and age disappears from the environment.

1:40 Then remove Black, the next value. It is gone too.

1:55 To remove many values at once, list them inside the brackets, separated by commas: City, demographic, Detail, location, and name.

2:15 Run it, and the environment says: Environment is empty. A fresh start!

2:30 I add a line of hash signs, to separate this part of the script. Next, we will install the packages we need. Nice and tidy!

R FOR DATA ANALYSIS · GETTING READY · LESSON 2

07 Install and load packages

Add extra tools to R with install.packages(), then switch them on with library().

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Install a package once with install.packages()
- Load a package in each session with library()
- Read loading messages without worry

The R code in this lesson

```
# Install once
install.packages("tidyverse")
install.packages("readxl")
install.packages("dplyr")

# Load every time you start R
library("tidyverse")
library("readxl")
library("dplyr")

# Where is R looking for files?
getwd()
```

What you will hear (times count from the end of the intro)

Intro R comes with many tools, but the best ones live in packages that we add ourselves. In this lesson, you will learn to install packages, load them each time you work, and read the messages R shows you along the way. Let's set up our toolbox.

0:00 Here is our list. First, a note: installing a package. Under it, install dot packages, with the package name in quotes: tidyverse, Hmisc, corrplot, readxl, openxlsx, stargazer, dplyr, expss, and formattable.

0:22 You only need to install a package once on your computer. It is like installing an app on your phone.

0:32 But each time you open R, you need to load the packages you want to use. For that, we use library, with the package name inside.

0:45 I run the library lines one by one. The console fills with red text. Please don't panic. Most of it is just information, like: the following objects are masked.

1:00 Masked means two packages have a function with the same name, so R tells you which one it will use. It is not an error.

1:15 One message did say error: there is no package called colorspace. That means one package needs another one. Installing the missing package fixes it.

1:40 So remember: red text is not always an error. Read it calmly, and look for the word Error.

2:08 Now, a new question: where does R look for our files? The function getwd, get working directory, tells us. Run it, and R shows the folder path.

2:23 Here I open the folder on the computer, to see the session folders where our files are saved.

2:42 Next, setwd, set working directory, tells R which folder to use. We will do that carefully in the next lesson. Great progress!

R FOR DATA ANALYSIS · GETTING READY · LESSON 3

08 Set your working directory

Tell R which folder holds your files, and fix the cannot change working directory error.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Check the current folder with getwd()
- Set a new folder with setwd()
- Fix path errors step by step

The R code in this lesson

```
# Where is R looking now?
getwd()

# Point R to your project folder (use your own path)
setwd("/Users/yourname/Desktop/R Teaching/2. Session Two")

# Check again
getwd()

# Tip: in RStudio, Session > Set Working Directory > Choose Directory also works
```

What you will hear (times count from the end of the intro)

Intro Before R can open a file, it needs to know where to look. In this lesson, you will learn to check your working directory, set a new one, and calmly fix the error that appears when the path is not quite right. Let's find our folder together.

0:00 We start with getwd. It shows the folder R is using now: the R Teaching folder on the desktop.

0:10 But our data file is inside the Session Two folder. So we use setwd, and inside the brackets, in quotes, we write the full path to that folder.

0:25 I type the path carefully: Users, my user name, Desktop, R Teaching, and Session Two.

0:45 Run it, and R says: Error, cannot change working directory. That means the path does not match a real folder exactly. Even one wrong character, or a missing space, will cause this.

1:12 Here I open the folder on the computer to check the exact folder name. The folder is called two, dot, space, Session Two.

1:25 I try again with double slashes, and a few small changes. Each time, I read the message, compare the path, and fix one thing.

1:50 And this time it works. getwd now shows the Session Two folder. In the Files pane, we can see our data file inside it.

2:30 I add clear notes in the script: understanding the working directory, and setting the working directory. Notes like these help you, and anyone who reads your script later.

2:50 Tip: if typing the path is hard, RStudio can do it for you. Use the Session menu, then Set Working Directory. Next, we will open our Excel file. Well done!

R FOR DATA ANALYSIS · GETTING READY · LESSON 4

09 Open an Excel file and inspect it

Load an Excel sheet with read_excel(), then look at it with head, str, summary and colnames.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Load an Excel file with read_excel()
- Inspect a dataset with head(), str() and summary()
- List column names with colnames()

The R code in this lesson

```
library(readxl)

# Opening an Excel file
Dataset <- read_excel("Clean_Dataset.xlsx")

# Inspecting the dataset
head(Dataset)       # first 6 rows
str(Dataset)        # structure: column types
unique(Dataset)     # unique rows
summary(Dataset)    # min, max, mean for numbers
colnames(Dataset)   # all column names
print(Dataset)      # print the table
```

What you will hear (times count from the end of the intro)

Intro Now the exciting part: real data! In this lesson, you will learn to open an Excel file in R, and then look at it from different angles, so you truly understand it before any analysis. Let's open our first dataset.

0:00 Our working directory is set. Now I write a note: opening an Excel file. The line is: Dataset, arrow, read underscore excel, and the file name in quotes: Clean underscore Dataset dot x l s x.

0:12 Run it. In the Environment pane, a new item appears: Dataset, with one hundred thirty five observations of twenty four variables. That means one hundred thirty five rows, and twenty four columns.

0:25 Now let's look inside. First, head. Head shows the first six rows in the console: province, district, sex, marital status, household size, and the survey answers.

0:45 Next, str, for structure. It lists every column, and its type: chr for text, and num for numbers. This is the best way to check that each column has the right type.

1:15 Unique shows the unique rows, and summary gives quick statistics for each column.

1:35 For number columns, summary shows the minimum, the first quartile, the median, the mean, the third quartile, and the maximum. For household size, the average is about nineteen.

2:10 Then colnames, which lists all the column names. Here we see the long survey question names, like, did you receive fifty kilograms of urea?

2:35 And finally print, which prints the table in the console. I add a note above these lines: inspecting dataset.

2:50 Head, str, summary, and colnames. Use them every time you open new data. They are your first look, and your first check. Wonderful work!

10 The pipe and select(): build a smaller table

Use the pipe %>% and select() to keep only the columns you need.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Type the pipe with Control, Shift and M
- Select columns into a new data frame
- Split long code over several lines

The R code in this lesson

```
library(dplyr)

# Pipe shortcut: Ctrl + Shift + M  (Mac: Cmd + Shift + M)
demographic <- Dataset %>% select('0.1 Province Name', '0.2 District Name',
                                  '0.11 Sex of household head',
                                  '0.12 Marital Status of household head',
                                  'HH Size')
View(demographic)
```

What you will hear (times count from the end of the intro)

Intro Big datasets can feel overwhelming. The trick is to keep only what you need. In this lesson, you will meet the pipe, one of the most loved tools in R, and use select to build a smaller, focused table. Let's make our data friendlier.

0:00 Here are two helpful shortcuts, written as notes. The pipe operator, percent, greater than, percent, has a shortcut: Control, Shift and M. And Shift and Enter helps when a command is long.

0:20 The pipe means: take this, and then do that. So Dataset, pipe, select, means: take the Dataset, and then select some columns.

0:40 Inside select, I list the columns I want, each in quotes: province name, district name, sex of household head, marital status, and household size.

1:02 Long lines are hard to read. So I press Enter after each comma, and RStudio keeps the code neatly lined up. R still reads it as one command.

1:25 At the front, I write: demographic, arrow. So the result is saved as a new data frame, called demographic.

1:45 Run it. In the Environment, demographic appears, with one hundred thirty five rows, and five columns. Same people, fewer columns.

1:52 Click on it, or use View, to open it like a spreadsheet. Here are our five columns: province, district, sex, marital status, and household size.

2:10 Look how much easier this is to read than the full dataset. Below in the script, I have also prepared the next steps: removing columns, and removing rows.

2:40 Take, and then do. That is the pipe. Once you get used to it, you will use it every day. Well done!

11 Remove columns and rows

Drop columns with a minus sign in select(), and drop rows by their number.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Remove one or two columns with select(-)
- Remove a row with [-135,]
- Remove several rows with c()

The R code in this lesson

```
# Remove one column
demographic <- demographic %>% select(-`0.2 District Name`)

# Remove two columns
demographic <- demographic %>% select(-`0.2 District Name`, -`0.11 Sex of household
head`)

# Remove one row (row 135)
demographic <- demographic[-135, ]

# Remove several rows
demographic <- demographic[-c(131, 132, 133, 134), ]
```

What you will hear (times count from the end of the intro)

Intro Sometimes the easiest way to clean data is to take things away. In this lesson, you will learn to remove columns with a simple minus sign, and to remove rows by their number, one or several at a time. Let's tidy up our table.

0:00 Our demographic table has five columns. Under the note, to remove a column from a data frame, we use select again, but with a minus sign before the column name.

0:22 Demographic, pipe, select, minus, district name. The minus means: everything except this column.

0:43 Run it. In the Environment, demographic now has four variables. The district column is gone.

1:04 Next, I remove the sex of household head column the same way. Now we have three variables.

1:25 To remove two columns at once, put both inside select, each with a minus sign, separated by a comma. First, I recreate the full table, so we can try it again.

1:50 Now rows. Rows are removed with square brackets. Demographic, open bracket, minus one hundred thirty five, comma, close bracket. The number before the comma is the row. Nothing after the comma means: keep all columns.

2:05 Before running it, let's look at the table. The last row is number one hundred thirty five.

2:20 Run it, and the Environment shows one hundred thirty four observations. Row one hundred thirty five is gone.

2:45 For several rows, use c inside the brackets: minus c, one hundred thirty one, one hundred thirty two, and so on. Be careful to close every bracket.

3:05 Minus means remove. For columns, inside select. For rows, inside square brackets. You've got this!

12 Rename columns, two ways

Give columns short, clear names with names() or with rename().

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Rename all columns at once with names()
- Rename chosen columns with rename()
- Write new name = old name correctly

The R code in this lesson

```
# Way 1: replace all names, in order
names(demographic) <- c('Province', 'District', 'Gender', 'Marital_Status',
'HH_Size')

# Way 2: rename(new name = old name)
demographic <- demographic %>% rename("Province" = '0.1 Province Name',
                                      "District" = '0.2 District Name',
                                      "Gender" = '0.11 Sex of household head',
                                      "Marital_Status" = '0.12 Marital Status of
household head',
                                      "HH_Size" = 'HH Size')
```

What you will hear (times count from the end of the intro)

Intro Long column names like zero point eleven, sex of household head, are hard to type and easy to get wrong. In this lesson, you will learn two ways to give your columns short, clear names. Let's make our table easier to work with.

0:00 Under the note, to rename a column name, the first way uses the names function. Names of demographic, arrow, c, and then the new names in quotes, in the same order as the columns.

0:15 Province, District, Gender, Marital underscore Status, and H H underscore Size. We use underscores instead of spaces. Names without spaces are much easier to use in R.

0:33 Run it, and open the table. Look at the headers: Province, District, Gender, Marital Status, H H Size. Clean and short!

0:50 But be careful: names replaces all the names in order. If you get the order wrong, the names will be on the wrong columns.

1:15 So here is the second way, which I often prefer. I write: Or. Then I recreate the table, so we start again from the original long names.

1:40 Demographic, pipe, rename. Inside rename, we write the new name, an equals sign, and then the old name. New name first, old name second.

2:10 Province equals zero point one province name. District equals zero point two district name. Gender equals sex of household head.

2:30 Marital status, and household size, the same way. IntelliSense helps me pick each column, so the old names are spelled exactly right.

3:00 Rename only touches the columns you list, so it is safer. And the code tells everyone exactly what changed. Excellent work!

13 Sort rows with arrange()

Order your table by one column, A to Z or Z to A, with arrange() and desc().

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Sort rows with arrange()
- Sort in reverse with desc()
- Check the order in the table view

The R code in this lesson

```
# Arranging rows: ascending (A to Z)
demographic <- demographic %>% arrange(Province)
demographic <- demographic %>% arrange(Gender)

# Descending (Z to A)
demographic <- demographic %>% arrange(desc(Gender))
```

What you will hear (times count from the end of the intro)

Intro A sorted table is much easier to read and check. In this lesson, you will learn to sort your rows with arrange, from A to Z, and from Z to A. It only takes one short line. Let's put our data in order.

0:00 Here is our table, with the new short column names. Right now, the rows are in the order they came from the survey.

0:25 I write a note: arranging rows, we can ascend or descend the rows. Ascend means from small to big, or A to Z. Descend means the opposite.

0:45 Demographic, pipe, arrange, and inside the brackets, the column to sort by: Province.

1:10 Run it. Now the provinces are in alphabetical order: Badakhshan first, then Farah, and so on.

1:25 Next, arrange by Gender. Run it, and open the table. Female rows come first, because F comes before M in the alphabet.

1:50 To reverse the order, wrap the column in desc, short for descending. Arrange, desc, Gender.

2:20 Run it, and now Male comes first. The same works for numbers: arrange household size, and the smallest families come first. With desc, the largest come first.

2:45 Arrange, and desc for reverse. Simple, and very useful when you check your data. Great job today!

14 Filter rows with filter()

Keep only the rows you need, such as male-headed or female-headed households.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Keep matching rows with filter()
- Use == inside filter, not =
- Save each filtered table with its own name

The R code in this lesson

```
# Filtering a data frame
Male_Only_Demographic <- demographic %>% filter(Gender == "Male")
Female_Only_Demographic <- demographic %>% filter(Gender == "Female")

View(Male_Only_Demographic)
```

What you will hear (times count from the end of the intro)

Intro Filtering is one of the most important skills in data analysis. In this lesson, you will learn to keep only the rows you need with filter, and you will see a very common mistake and how R kindly tells you the fix. Let's filter together.

0:00 I write a note: filtering a data frame. Our goal is a table with only male household heads.

0:10 Male Only Demographic, arrow, demographic, pipe, filter. Inside filter, the condition: Gender equals Male, in quotes.

0:30 Run it. And here is an error. But read it slowly, because R is helping us. It says: we detected a named input. This usually means you used equals instead of double equals. Did you mean Gender, double equals, Male?

0:55 Remember our comparison lesson? One equals sign assigns. Two equals signs compare. Inside filter, we are asking a question, so we need two.

1:25 I add the second equals sign, and run it again. It works! A new table appears in the Environment: one hundred thirty three rows. Only male household heads.

1:40 Let's open it with View, and check the Gender column. Every row says Male. Perfect.

2:10 Now the same for women. I copy the line, change the name to Female Only Demographic, and the value to Female.

2:25 Run it. This table has two rows. So in this sample, two households are headed by women. Filtering helps us find these answers quickly.

2:45 Next, I write a note: complex filter. In the next lesson, we will filter with several conditions at once. Well done!

15 Filter with several conditions

Combine conditions in one filter() to find exactly the rows you are looking for.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Add several conditions separated by commas
- Read zero rows as an answer, not an error
- Change a condition and compare the results

The R code in this lesson

```
# Complex filter: all conditions must be TRUE
DeepFilter <- demographic %>% filter(Gender == "Male",
                                     Province == "Badakhshan",
                                     Marital_Status == "Single")
View(DeepFilter)
```

What you will hear (times count from the end of the intro)

Intro Real questions often have more than one part. In this lesson, you will learn to filter with several conditions at once, such as gender, province and marital status together, and you will learn what it means when a filter returns no rows. Let's dig a little deeper.

0:00 Under the note, complex filter, I start a new table called Deep Filter.

0:15 Demographic, pipe, filter. The first condition: Gender, double equals, Male.

0:32 Then a comma, and the second condition: Province, double equals, Badakhshan. In a filter, a comma means and. Every condition must be true.

0:52 A third condition on a new line: Marital Status, double equals, Single.

1:15 Run it, and look at the Environment: Deep Filter, zero observations. Is that an error? No! It is an answer. In this data, there is no row that matches all three conditions as I first typed them.

1:35 Let's change one condition, from Single to Married, and run it again. Now we get a different result. Changing one condition at a time is a great way to explore your data.

1:52 And here, with Single, three rows match: three single men in Badakhshan. View shows them clearly.

2:20 Notice how each condition narrows the table a little more. First, all men. Then, men in Badakhshan. Then, single men in Badakhshan.

> **2:48** Next, I add a new note: converting numerical values to numeric. We will learn about converting data types soon. Excellent work!

16 Combine tables with bind_cols() and bind_rows()

Join tables side by side, or stack them one under another.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Add columns from another table with bind_cols()
- Stack rows from another table with bind_rows()
- Spot NA where columns do not match

The R code in this lesson

```
Main_Question <- Dataset %>% select('3. Did you receive 50 KG UREA?',
                                    '4. If not, how many KGs did you receive?',
                                    '10. How was the physical quality of the wheat
seed received?')

# Combining data frames: column-wise (same number of rows)
demographic <- bind_cols(demographic, Main_Question)

# Combining data frames: row-wise (stacks rows)
demographic <- bind_rows(demographic, Initial_Information)
```

What you will hear (times count from the end of the intro)

Intro Data often comes in pieces. In this lesson, you will learn to put the pieces together: side by side with bind columns, or one under another with bind rows. You will also see what NA means when the pieces don't match perfectly. Let's build a bigger picture.

0:00 We have three tables now. Demographic, with five columns. Initial Information, also with five columns. And Main Question, with three survey questions: urea, kilograms received, and seed quality.

0:20 Let's look at demographic first. One hundred thirty five rows, five columns.

0:36 I write a note: combining data frames, column-wise. That means side by side.

1:00 Demographic, arrow, bind underscore cols, and inside, demographic and Main Question. The tooltip helps us with the arguments.

1:12 Run it. Demographic now has eight variables. Open it, and you see the three question columns added on the right. Same rows, more columns.

1:40 Bind columns only works well when both tables have the same rows, in the same order. Here, both come from the same dataset, so they match.

1:52 Now row-wise. Demographic, arrow, bind underscore rows, with demographic and Initial Information.

2:05 Run it. Now we have two hundred seventy rows. The second table was stacked underneath.

2:25 Scroll down, and look at the question columns. In the lower rows, they say NA. NA means not available. Initial Information did not have those columns, so R leaves them empty.

2:55 Bind columns adds columns, bind rows adds rows. And when you see NA, ask yourself: is something missing, or did the tables just not match? Very well done!

17 Recode labels to numbers with if_else()

Turn Male and Female into 1 and 2, and back again, with mutate() and if_else().

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Change a column with mutate()
- Recode two choices with if_else()
- Convert codes back to labels

The R code in this lesson

```
# Converting a two-choice variable from label to value
demographic <- demographic %>% mutate(Gender = if_else(Gender == "Male", 1, 2))

# Converting from value back to label
demographic <- demographic %>% mutate(Gender = if_else(Gender == 1, "Male",
"Female"))
```

What you will hear (times count from the end of the intro)

Intro Many analyses need codes instead of words, like one for male and two for female. In this lesson, you will learn to recode a column with mutate and if else, and then turn the codes back into clear labels. Let's practice recoding together.

0:00 We start from a fresh demographic table with short column names. In the Gender column, we see the words Male and Female.

0:35 I write two notes. Converting a two choices variable from character to numeric. And converting it back, from numeric to character.

0:45 The tool for changing a column is mutate. Demographic, arrow, demographic, pipe, mutate. Inside mutate, we say which column to change: Gender equals.

1:05 Then if underscore else. It needs three parts: a condition, the value if true, and the value if false. Just like IF in Excel.

1:30 The condition: Gender, double equals, Male. If true: one. If false: two.

1:50 Run it, and open the table. The Gender column now shows one and two. One for male, two for female.

2:15 Now let's go back. Same pattern: mutate, Gender equals, if else. But this time the condition is: Gender, double equals, one.

2:40 If true: Male, in quotes. If false: Female, in quotes. Text needs quotes, numbers don't.

3:00 Run it, and the labels are back. Mutate changes a column, and if else chooses between two values. A powerful pair! Great work.

18 Group numbers into ranges with cut()

Turn household size into groups like 1 to 5 and 5 to 10, then count them with cross_cases().

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Create ranges with cut(), breaks and labels
- Use Inf for an open top range
- Count each range with cross_cases()

The R code in this lesson

```
library(expss)

# Make ranges from a number column
demographic <- demographic %>% mutate(HH_Size = cut(HH_Size,
                      breaks = c(1, 5, 10, 15, 20, Inf),
                      labels = c("1 to 5", "5 to 10", "10 to 15",
                                 "15 to 20", "More than 20")))

# Frequency table
cross_cases(demographic, cell_vars = HH_Size)
```

What you will hear (times count from the end of the intro)

Intro A long list of numbers is hard to read. Groups are much easier: small, medium, and large families, for example. In this lesson, you will learn to group numbers into ranges with cut, and then count each group in a neat table. Let's make our numbers tell a story.

0:00 On the right, the Viewer shows a frequency table of household size. Every single size has its own row: three, four, five, seven, and so on. It is long and hard to read.

0:10 I write a note: to make a range of column in a dataset. Then demographic, pipe, mutate.

0:30 Inside mutate: H H Size equals cut. Cut takes the number column, and cuts it into pieces.

0:45 First, breaks. The breaks are the edges of each group: one, five, ten, fifteen, twenty, and Inf.

1:00 Inf means infinity. It makes the last group open at the top, so every big family is included, no matter how large.

1:15 Then labels, one for each group, in quotes: one to five, five to ten, ten to fifteen, fifteen to twenty, and more than twenty.

1:45 Be careful with the brackets. There are several levels here, so close each one, slowly.

2:40 Run it, and open the table. The household size column now shows the group names instead of numbers.

2:55 Now let's count them. Cross underscore cases, from the expss package, with cell vars equals H H Size.

3:12 And here is our new table. Twenty four households have one to five members, and fifty seven have more than twenty. Much clearer! Wonderful work.

R FOR DATA ANALYSIS · EDITING DATA · LESSON 1

19 A new dataset: load, view and select

Start a new project script, load a fresh dataset, view it, and select the columns you need.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Start a script with setwd(), library() and read_excel()
- Open a dataset with View()
- Select columns by name into a new data frame

The R code in this lesson

```
setwd("/Users/yourname/Desktop/R Teaching/6. Session Six")
library(readxl)
library(tidyverse)
library(dplyr)

Dataset <- read_excel("Dataset.xlsx")

# To view a dataset
View(Dataset)

# Select columns to create a data frame
Demographic <- Dataset %>% select("Province Name", "Sex of household head",
                                  "Marital Status of household head")
```

What you will hear (times count from the end of the intro)

Intro Today we start with a new dataset. In this lesson, you will see how to begin a script the right way, load the data, view it like a spreadsheet, and select the columns you need. It is a review of earlier skills, so you can feel how much you already know.

0:00 A good script starts with three things at the top: setwd, to set the folder; the library lines, to load our packages; and read excel, to load the data. Here they are.

0:12 Run them, and the Environment shows Dataset: three hundred fifty two observations, of twenty variables. A bigger dataset than before.

0:25 I write a note: to view a dataset. Then View, with a capital V, and the dataset name inside.

0:40 View opens the data in a new tab, like a spreadsheet. You can scroll, sort, and look at every column.

1:10 Next note: to select a column or columns from a dataset, or to create a data frame.

1:40 When I reload the packages, some red messages appear, like: there is no package called colorspace. As we learned, this means a supporting package is missing, and installing it fixes the problem.

2:05 Now the select line. Demographic, arrow, Dataset, pipe, select. Inside, the column names in quotes: Province Name, and Sex of household head.

2:25 In the Environment, click the small arrow next to Dataset, and you can see every column name. This helps you copy the names exactly.

2:40 I add a third column: Marital Status of household head. Run it, and Demographic appears: three hundred fifty two rows, and three columns.

2:58 Setup, view, and select. That is how almost every analysis begins. You are getting faster!

R FOR DATA ANALYSIS · EDITING DATA · LESSON 2

20 Select columns by position with [,]

Pick columns by their number with square brackets, and use ranges like 6:10.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Read the [rows, columns] pattern
- Select columns by number with c()
- Use a colon for a range of columns

The R code in this lesson

```
# [rows, columns]: empty rows = all rows
Demographic_1 <- Dataset[, c(2, 3, 4)]

# A colon makes a range: 6:10 means 6, 7, 8, 9, 10
Main <- Dataset[, c(2, 3, 6:10)]

View(Main)

# Count values in a column
table(Demographic$`Sex of household head`)
```

What you will hear (times count from the end of the intro)

Intro Typing long column names can be tiring. In this lesson, you will learn a shortcut: selecting columns by their position number, with square brackets. You will also learn the colon, which picks a whole range at once. Let's try it.

0:00 Here is the dataset in View. Column one is Single Response, column two is Province Name, three is Sex of household head, and four is Marital Status.

0:20 Square brackets work like an address: first the rows, then a comma, then the columns. If we leave the rows empty, R takes all the rows.

0:35 So: Demographic underscore one, arrow, Dataset, open square bracket, comma, c, two, three, four, close bracket.

0:55 Run it, and the new table has three hundred fifty two rows, and three columns: province, sex, and marital status. The same result as select, with much less typing.

1:15 Now a second table, called Main. Inside c, I write two, three, and then six, colon, ten.

1:35 The colon means: from six to ten. So we get columns six, seven, eight, nine, and ten, without typing each one.

1:50 Run it. Main has seven columns. Open it, and there they are: province, sex, the land questions, and the wage labor answers.

2:30 A small warning: positions change if columns are added or removed. Names are safer for scripts you will reuse. Positions are great for quick looks.

2:50 Next, a new note: to bring changes in a single or more values or cells. Before we change anything, I use table to count the values in the Sex of household head column. Nice work!

21 Check counts with table() and remove rows safely

Count values with table(), remove a row, and learn why the comma inside [ ] matters so much.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Count each value with table()
- Remove a row with [-1,]
- Avoid the missing comma mistake

The R code in this lesson

```
# Count the values in one column
table(Demographic$`Sex of household head`)

# Remove the first row (note the comma!)
Demographic <- Demographic[-1, ]

# Without the comma, R removes a COLUMN instead
# Demographic <- Demographic[-1]   # careful!

# Remove a specific row
Demographic <- Demographic[-112, ]
```

What you will hear (times count from the end of the intro)

Intro Before we change any data, we should always check it first. In this lesson, you will learn to count values with table, remove rows safely, and avoid one tiny mistake, a missing comma, that can change your data in a way you did not expect. Let's be careful together.

0:00 First, some tidying. I remove the extra tables we made, Demographic one and Main, so the Environment is clear.

0:15 Under the note, to bring changes in a single or more cells, I start with table. Table of Demographic, dollar sign, Sex of household head.

0:22 The dollar sign picks one column from a table. Run it, and the console shows: Female one, Male three hundred fifty one.

0:55 Let's view the table. In row one, the household head is female, a widow from Khost. Let's practice removing that row.

1:15 I type Demographic, open square bracket, minus one, close bracket. But look: I forgot the comma! Without the comma, R removed the first column, not the first row. Only one observation of three variables is shown. Something is wrong.

1:40 So I recreate the table, and this time I write minus one, then a comma. Now R knows the one is a row.

2:05 Run it, and the table has three hundred fifty one rows. Table now shows Male three hundred fifty, and no female. Exactly one row is gone.

2:30 Every time you remove a row, run table again, and check that the numbers changed the way you expected.

2:50 Here I remove row one hundred twelve. Three hundred forty eight rows are left. Check, change, check again. That is a very good habit. Well done!

22 Change one cell, or many cells at once

Replace values with [row, column] <- value, for a single cell or a whole range of rows.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Change one cell with [row, column]
- Change a range of rows with 1:200
- Check every change with table()

The R code in this lesson

```
# Change one cell: row 115, column 2
Demographic[115, 2] <- "Male"

# Change many cells: rows 1 to 200, column 2
Demographic[1:200, 2] <- "Female"

# Check the result
table(Demographic$`Sex of household head`)

# Undo by setting them back
Demographic[1:352, 2] <- "Male"
```

What you will hear (times count from the end of the intro)

Intro Sometimes you need to correct a value, like a typing mistake from the field. In this lesson, you will learn to change one cell, or many cells at once, using the row and column address, and to check every change with table. Let's correct some data carefully.

0:00 We start again from the full Demographic table: three hundred fifty two rows. Table shows one female and three hundred fifty one males.

0:12 To change one cell, we use its address: Demographic, square brackets, row one hundred fifteen, comma, column two. Then the arrow, and the new value: Male, in quotes.

0:30 Run it, then run table again. Now it says Male, three hundred fifty two. The one female value has changed.

0:45 To change many cells, use a range of rows. Demographic, rows one colon two hundred, column two, arrow, Female.

1:00 Run it, and view the table. The first two hundred rows now say Female. Table shows two hundred female, and one hundred fifty two male.

1:20 This is just for practice, of course. In real work, you would only change a value when you know it is wrong, for example after checking with the field team.

1:35 Let's set them back: rows one to two hundred, column two, Male. Run table again to check.

2:05 And here I change the single cell, row one hundred fifteen, back to Female. Table shows one female and three hundred fifty one male again, just like at the start.

2:45 Finally, rows one to three hundred fifty two: every row. Use ranges like this with care. Always check with table after each change. Excellent work today!

R FOR DATA ANALYSIS · EDITING DATA · LESSON 5

23 attach(): type column names with ease

Attach a dataset so RStudio suggests its column names, and understand the masking messages.

BY THE END OF THIS SECTION YOU WILL BE ABLE TO

- Attach a dataset with attach()
- Use autocomplete to pick column names
- Understand the objects are masked message

The R code in this lesson

```
# Attaching a dataset: it eases selecting column names
attach(Dataset)
attach(Demographic)

# Now RStudio suggests column names as you type
Demographic_1 <- Dataset %>% select(`Province Name`, `Sex of household head`,
                                    `Marital Status of household head`)

# When you are done
detach(Demographic)
```

What you will hear (times count from the end of the intro)

Intro Typing long column names exactly right can be hard. In this lesson, you will learn attach, which lets RStudio suggest column names as you type. You will also learn what the masking messages mean, so they never worry you. Let's make typing easier.

0:00 Here is our script so far. I add a new note: attaching a dataset. It eases selecting column names.

0:12 The function is attach. Attach Dataset, and then attach Demographic.

0:25 Run them. The console shows some red text: the following objects are masked. Don't worry. It means both tables have columns with the same names, so R tells us which one comes first.

0:45 Now let's see why attach helps. I write a new select line: Demographic underscore one, arrow, Dataset, pipe, select.

1:20 Inside select, I start typing: provin. RStudio shows a list: Province Name, from Dataset. I just press Enter, and the full name is written for me.

1:50 No spelling mistakes, no missing spaces. This saves a lot of time with long survey column names.

2:05 I add Sex of household head, and Marital Status of household head, the same way.

2:40 Run it, and Demographic one appears in the Environment: three hundred fifty two rows, three columns.

2:48 One tip: attach is helpful while you explore, but it can cause confusion if you attach many tables. When you are done, use detach. That brings us to the end of our R sessions. You have learned so much. Thank you, and keep practicing!